

Deep Learning Architectures for Time Series: The Mathematics of Temporal Memory

by

STAVROS ZOIS DIMOU

Under the guidance of

DR. OURANIA THEODOSIADOU



DEPARTMENT OF MATHEMATICS
ARISTOTLE UNIVERSITY OF THESSALONIKI

June, 2026

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Ms. Ourania Theodosiadou. Your mentorship, support, and kindness since the very first day of working together have been invaluable. Your willingness and constant availability throughout this project sparked my curiosity for time series forecasting and served as a massive launchpad for my career in data science.

I also would like to thank my high school mathematics tutors, Mr. Thukidides Stokas and the late Mr. Alexandros Stokas. You were my first true mentors and a massive reason for me enrolling in the university and choosing the path of a mathematician. Thank you for your profound impact on my life. Affectionately dedicated to the memory of Alexandros Stokas, whose lessons kept expanding the spark I had for math.

To my friends Teo, Giannis, Timos, Mike, Periklis, Kiriakos, Maria thank you for the endless support and love, This journey would have been incredibly lonely without you.

Finally, my deepest love and thanks go to Marilena. Thank you for your infinite patience, your unwavering belief in me, and for being my safe harbor. Your support made all the difference.

Stavros Zois Dimou

Table of Contents

Acknowledgement	i
Table of Contents	ii
List of Figures	iv
List of Tables	iv
1 The Root RNN Derivation	1
1.1 Roots in Dynamic System and Signal Processing Theory.	1
1.1.1 Model outline	1
1.2 The RNN as a numerical solution of the DE	2
1.2.1 Euler Approximation Method	2
1.2.2 Deriving the RNN from Euler	3
1.2.3 Stability of the RNN System	4
2 Unrolling in Time: From Recurrence to Sequence	7
2.1 What is unrolling and why?	7
2.1.1 The RNN recursive/rolled unit	8
2.1.2 The RNN unit unrolled	9
2.2 Setting Up for Training	9
2.2.1 Dataset Segmentation	10
2.2.2 The Unrolling proposition	11
3 Training the RNN: Gradients and Adversarial Behaviors	14
3.1 The error function and main gradients	14
3.2 Backwards DP	15
3.2.1 The Recursive Equations	16
4 How LSTMs Learn to Remember	19
4.1 The LSTM as a solution	19
4.2 The LSTM unit architecture	19
4.2.1 The sub-units of the LSTM unit	19

4.2.2	A new memory	20
4.2.3	The gates	20
5	Empirical Validation - Proof of Concept	22
5.1	Introduction and Experimental Setup	22
5.2	Case Study I: The Long-Window Trap in Real-World Time Series	22
5.2.1	Experimental Design	22
5.2.2	Results and Discussion	23
5.3	Case Study II: The Adding Problem	24
5.3.1	The Data Generating Process	25
5.3.2	Breaking the Plateau	25
5.4	Conclusion	26
	Bibliography	27

List of Figures

2.1	The cyclic computational graph of an RNN cell at a single discrete time step n	9
2.2	The unrolled Directed Acyclic Graph (DAG) representation spanning multiple sequential time-steps.	9
4.1	Detailed internal schematic of the Long Short-Term Memory (LSTM) block, mapping the structural interaction of the forget, input, and output gates with the long-term cell state rail.	21
5.1	Validation MAE over epochs. Notice the initial convergence speed of the SimpleRNN compared to the LSTM.	23
5.2	Autocorrelation Function (ACF) of Melbourne Daily Minimum Temperatures.	24
5.3	Partial Autocorrelation Function (PACF) of Melbourne Daily Minimum Temperatures.	24
5.4	Validation MAE on the Adding Problem ($L = 200$). The SimpleRNN remains trapped at baseline error, while the LSTM successfully synchronizes its gates to solve the sequence.	25

List of Tables

2.1	Segment-wise representation via masking function $\vec{w}_m[n]$	11
5.1	Performance metrics for the 365-day lookback temperature forecasting task.	23

Chapter 1

The Root RNN Derivation

1.1 Roots in Dynamic System and Signal Processing Theory.

In this section, we aim to outline the system and signal theory problem that stands at the root of the formulation of the RNN architecture, through following the methodological framework presented in [1].

1.1.1 Model outline

We begin by introducing these two main signals (d-dimensional time functions) with their definitions and corresponding notation.

- $\vec{s}(t)$: The state signal which models the internal dynamics and memory of the system.
- $\vec{x}(t)$: The input signal which models the external features (external stimuli - perturbations) that are presented to the system.

Consider now the main ODE that describes the system,

$$\frac{d\vec{s}(t)}{dt} = \vec{h}(\vec{x}(t), \vec{s}(t)) + \vec{\phi} \quad (1.1)$$

where $\vec{h}$ is also a d-dimensional valued vector function and $\vec{\phi}$ is a constant term.

Intuitively from this model we can see that the way that the memory changes with regards to itself (past memory) and external input - this is crucial to setting up the RNN models that are known for processing sequential data that utilizes both memory and input (an example in NLP is the fact that introducing one new word to a sentence can change it's holistic meaning).

A simple and yet effective framework for the function $\vec{h}$ is what is the 'additive model' which relates the signals through vector addition. We need to define the following first.

Definition 1.1.1. We define the readout signal vector $\vec{r}$ as $\vec{r}(t) = G(\vec{s}(t))$ where G is an activation ¹(or warping) function.

Using this definition the readout signal, the **additive model** is formulated as follows.

$$\vec{h}(\vec{x}(t), \vec{s}(t)) = A\vec{s}(t) + B\vec{r}(t - t_0) + C\vec{x}(t) \quad (1.2)$$

Which transforms ² 1.1 to

$$\frac{d\vec{s}(t)}{dt} = A\vec{s}(t) + B\vec{r}(t - t_0) + C\vec{x}(t) + \vec{\phi} \quad (1.3)$$

where

- A, B, C denote $d \times d$ dimensional matrices (indicating a linear function of the respective signals)
- t_0 denotes a time delay - capturing model's ability to remember information from previous timestamps in the readout signal.

1.2 The RNN as a numerical solution of the DE

As we have mentioned, we will develop the traditional RNN as a natural consequence of numerically approximating the solutions of (1.3) through the Backwards Euler Approximation method ³. The intuition behind the choice of this specific method lies in the concept of approximating an analog-continuous time domain signal by a discretization of the the time dimension. This is what leads to the standard RNN definition which updates the hidden state at time $t + 1$ as a function of the hidden state at t .

1.2.1 Euler Approximation Method

We begin by outlining the steps of the **Backwards Euler Approximation** method.

Consider the following initial value problem:

$$\frac{ds(t)}{dt} = f(t, s(t)), \quad s(0) = s_0.$$

¹A warping or activation function G is a function that allows encoding of the signals behavior in more numerically stable ranges (e.g. the sigmoid function $\sigma(z) = \frac{1}{1+\exp(-z)}$ encodes all the real line $\mathbb{R}$ into $[0,1]$ while maintaining monotonicity) and introduces non-linearity. Additional discussion on the role of G can be found in [2] (different options) and in [1] (quasi-linearity, saturation and bipolarity). This transformation enables the readout signal to serve as a memory mechanism, capturing the essential dynamics of past inputs.

²With this model we can capture the internal dynamics of the system through $\vec{s}$ which determine the stability (more later). Additionally, the readout vector $\vec{r}$ provides access to temporally delayed representations of past states, enabling memory-like behavior.

³One might find a more formal derivation of the method in [3]

1. By the definition of the derivative, we can approximate:

$$\frac{ds(t)}{dt} \approx \frac{s(t + \Delta t) - s(t)}{\Delta t}.$$

2. Rearranging yields the update rule:

$$s(t + \Delta t) \approx s(t) + \Delta t \cdot \frac{ds}{dt}.$$

3. Let us define a discrete time grid $t_n = t_0 + n\Delta T$, where ΔT is the fixed time step.
4. In the Backward Euler method, we evaluate the derivative at the next time point, t_{n+1} , rather than at t_n . This gives the implicit update:

$$s(t_{n+1}) \approx s(t_n) + \Delta T \cdot f(t_{n+1}, s(t_{n+1})).$$

5. Introducing the notation $s[n] := s(t_n)$, the update becomes:

$$s[n + 1] \approx s[n] + \Delta T \cdot f(n + 1, s[n + 1]), \quad n \geq 0, \quad (1.4)$$

which is the iterative step of the Backward Euler method.

1.2.2 Deriving the RNN from Euler

We now apply the method to 1.3, with setting t_0 to be ΔT (this allows us to utilize the memory of the readout signal of the previous step n in the calculation of the state signal at time stamp $n + 1$)

So applying 1.4 to the 1.3 and denoting the $\approx$ with the equal sign yields:

$$\begin{aligned} \vec{s}[n + 1] &= \vec{s}[n] + \Delta T \cdot \left(A\vec{s}[n + 1] + \underbrace{B\vec{r}[(n + 1)\Delta T - \Delta T]}_{= B\vec{r}[n]} + C\vec{x}[n + 1] + \vec{\phi} \right) \\ &= \vec{s}[n] + \Delta T \cdot (A\vec{s}[n + 1] + B\vec{r}[n] + C\vec{x}[n + 1] + \vec{\phi}) \end{aligned}$$

So now by solving for $\vec{s}[n + 1]$ we obtain:

$$\underbrace{(I - (\Delta T) \cdot A)}_{W_s^{-1}} \vec{s}[n + 1] = \vec{s}[n] + \Delta T \cdot B\vec{r}[n] + \Delta T \cdot C\vec{x}[n + 1] + \Delta T \cdot \vec{\phi} \quad (1.5)$$

And denoting $W_s = [I - (\Delta T) \cdot A]^{-1}$ and multiplying both sides by W_s and making the index shift $n \rightarrow n - 1$ we get:

$$\vec{s}[n] = W_s \vec{s}[n-1] + \underbrace{(\Delta T \cdot W_s B)}_{W_r} \vec{r}[n-1] + \underbrace{(\Delta T \cdot W_s C)}_{W_x} \vec{x}[n] + \underbrace{(\Delta T \cdot W_s \vec{\phi})}_{\vec{\theta}_s} \quad (1.6)$$

And via defining the matrices and the vector outlined in the underbraces we obtain what is formally known as the **canonical RNN form**:

$$\vec{s}[n] = W_s \vec{s}[n-1] + W_r \vec{r}[n-1] + W_x \vec{x}[n] + \vec{\theta}_s \quad (1.7)$$

One might notice that the information that the RHS provides is composed out of the state and the readout signal at time $n-1$ and from the input at n . However - drawing from the definition of the readout signal as $\vec{r}(t) = G(\vec{s}(t))$ we see that the first component essentially comes from the state signal at $n-1$ through both terms.

We then can simplify our model by considering the weights and biases $W_s, W_r, \vec{\theta}_s$ as parameters that take on values independently from one another. So we choose W_s to be the zero matrix which gives us more convenient model that still carries the same information. ⁴

Although the original derivation in [1] does not immediately define an output function, it is customary in RNN architectures to include a linear transformation of the readout signal to produce the output. ⁵ We therefore define as following

Definition 1.2.1. We define the observable output signal of the system at time n to be the following linear transformation of the readout signal: $\vec{y}[n] = W_y \vec{r}[n] + \vec{\theta}_y$ where W_y and $\vec{\theta}_y$ are the output weight and bias respectively.

Now putting everything together we obtain the **standard RNN forward pass equations**

$$\vec{s}[n] = W_r \vec{r}[n-1] + W_x \vec{x}[n] + \vec{\theta}_s \quad (1.8)$$

$$\vec{r}[n] = G(\vec{s}[n]) \quad (1.9)$$

$$\vec{y}[n] = W_y \vec{r}[n] + \vec{\theta}_y \quad (1.10)$$

1.2.3 Stability of the RNN System

One of the most well-documented challenges in training recurrent neural networks (RNNs) is the phenomenon of *vanishing and exploding gradients*, particularly in the context of

⁴This way of working also further solidifies the notion that the readout signal is responsible for carrying the memory of the network.

⁵Again we are aiming to bridge the gap between signal processing and conventional ML

modeling long-term dependencies.

In this section, we give a high-level overview of a structural trade-off at the core of the RNN architecture. In Equation (1.8), we presented the standard state-update formulation of the RNN. From a systems-theoretic perspective, it is natural to examine the stability of this dynamical system.

A key insight as outlined in [1] is that the very condition that enforces the stability of the state dynamics is also the one responsible for the emergence of vanishing or exploding gradients during training.

Specifically, a necessary and sufficient condition for forward stability is that the eigenvalues μ_i of the recurrent matrix W_r satisfy: $0 < \mu_i < 1$

In Chapter 3, we will explore how this condition plays a dual role in the Backpropagation Through Time (BPTT) algorithm. We will show that even if the system is stable under this condition - implying $0 < \mu_i < 1$, gradients still vanish.⁶ Conversely, if even a single eigenvalue μ_i of W_r is greater than one then gradients can explode exponentially, leading to the well known exploding gradient problem.

Both cases are detrimental to the training of the network, as gradients that are too small or too large disrupt the behavior of gradient descent based algorithms, leading to ineffective learning due to either vanishing updates or unstable parameter jumps.

This highlights a fundamental tradeoff inherent in the RNN design balancing dynamic stability with gradient flow.

⁶This typically occurs when the argument to G is in the flat (saturated) region, resulting in $G'(\vec{s}) \approx 0$, and hence any backpropagated gradient is multiplied by a near-zero value.

-

Chapter 2

Unrolling in Time: From Recurrence to Sequence

2.1 What is unrolling and why?

In the previous chapter, we introduced the standard RNN equations (Equation 1.8). From this point forward, we will refer to these equations collectively as the *RNN cell*. The repeated computation of the RNN cell across time steps $n = 1, 2, 3, \dots$ given an initial state signal $s[0]$ is commonly referred to as *unrolling the RNN in time*.

Drawing from [1], we will illustrate the unrolling process with a focus on intuitive understanding. This perspective is crucial for adapting the classic backpropagation algorithm that originally designed for training feedforward neural networks into the form that is used for recurrent neural networks: *Backpropagation Through Time (BPTT)*. BPTT enables the RNN to be trained using a forward and backward pass structure that mirrors that of multilayer perceptrons (MLPs), while accounting for temporal dependencies.

Unrolling is what enables the RNN architecture to process inputs of varying length (e.g., sentences of different lengths or time series windows of different durations). As noted in [4], this is achieved not by having an input layer of variable size, but by constructing what appears to be a variable number of layers with each layer representing a timestep, and all share a single set of parameters $\Theta = \{W_r, W_x, \vec{\theta}_s\}$ across the entire network to be optimized for.¹

Although it may appear that the network contains many layers, they are in fact multiple applications of the same recurrent unit across time.

Let us now develop some visual intuition for what the unrolling process achieves.

¹For a more detailed discussion on why unrolling is necessary, see the explanation in [1], which presents the RNN as an approximation of an IIR system using a finite-length FIR structure.

2.1.1 The RNN recursive/rolled unit

Let us begin by initializing the state signal $s[0]$. We can then compute the values $s[1]$ and $r[1]$ using the RNN equations.

To build an intuition for this recursive process, imagine a camera that takes a snapshot at each time step n . Each snapshot captures the computation occurring at that moment, based on the input $x[n]$ and the state signal $s[n - 1]$ from the previous snapshot. With every time step, this process is repeated: the camera clicks, and the RNN cell updates the internal state.

Although the RNN processes sequences step by step, it retains only the information stored in the previous state $s[n - 1]$ and the current input $x[n]$. This recursive dynamic is what gives the RNN its memory-like behavior, despite having no explicit long-term memory storage beyond the state signal. This recursive formulation is both sound and technically correct. However, it presents two significant challenges:

1. **Training via gradient descent becomes non-trivial.** The recursive structure does not directly support the computation of gradients through time. In order to apply gradient descent effectively, we must use backward dynamic programming (i.e., backpropagation), which requires knowledge of the intermediate states at every time step. We will later see how unrolling the RNN addresses this issue by exposing the full computational graph.
2. **High computational cost.** Recursive computations are typically implemented using loops, which can be inefficient in practice—especially in high-level machine learning languages like Python. Unrolling the RNN allows us to reformulate these operations using vectorized expressions, which are substantially more efficient and leverage modern hardware acceleration.

This recursive structure can be visualized as a cyclic computational graph for each different time stamp. At each time step n , the RNN cell retrieves the previous state $s[n - 1]$ from the left (i.e., from the preceding time step) and receives the current input $x[n]$ from below. Together, these form the basis for computing the current state $s[n]$ and read-out $r[n]$. If left in their recursive form, the individual computations at each time step can appear as isolated cyclic graphs—each operating with access only to the immediately previous state and current input. In this view, the RNN’s memory is constrained to a strictly local scope. Intuitively these graphs do not live in the same ‘dimension’ with one another.

This recursive structure can be visualized as a cyclic computational graph for each different time stamp. At each time step n , the RNN cell retrieves the previous state $s[n - 1]$ from the left (i.e., from the preceding time step) and receives the current input $x[n]$ from below. Together, these form the basis for computing the current state $s[n]$ and read-

out $r[n]$. If left in their recursive form, the individual computations at each time step can appear as isolated cyclic graphs—each operating with access only to the immediately previous state and current input. In this view, the RNN’s memory is constrained to a strictly local scope. Intuitively these graphs do not live in the same ‘dimension’ with one another.

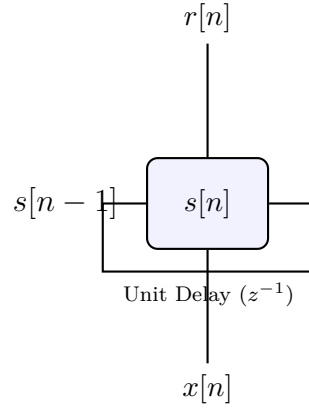


Figure 2.1: The cyclic computational graph of an RNN cell at a single discrete time step n .

2.1.2 The RNN unit unrolled

However, by unrolling the RNN over time, we conceptually “stitch together” these graphs into a single directed acyclic graph that spans the full sequence length of the input vector $\vec{x}[n]$ with $n \in \{1, \dots, T\}$. This transformation enables both the forward computation and backward gradient flow to traverse the entire sequence. The model now has access to all the signals in $\{1, \dots, T\}$ not just the previous one.

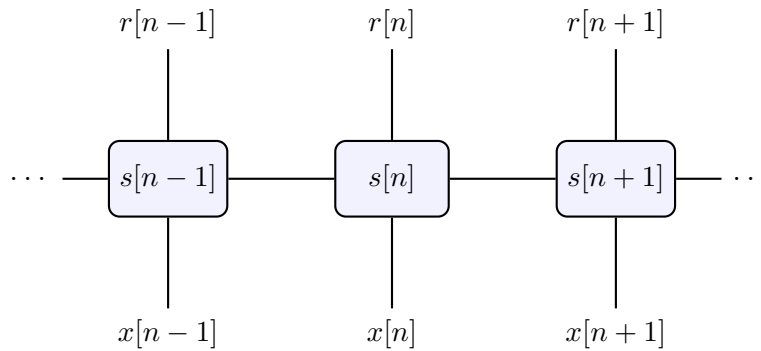


Figure 2.2: The unrolled Directed Acyclic Graph (DAG) representation spanning multiple sequential time-steps.

2.2 Setting Up for Training

Before diving into the Backpropagation Through Time (BPTT) algorithm, we introduce several foundational concepts that are essential for understanding how RNNs are trained as supervised learning models.

2.2.1 Dataset Segmentation

Since the RNN is trained using supervised learning, it requires a labeled dataset. In sequence modeling tasks, this typically means that each input sequence is paired with a corresponding sequence of ground truth labels.

Definition 2.2.1. Suppose we are given a dataset of length N . We denote the ground truth labels as a sequence $\vec{v} \equiv \vec{v}[n]$ for $n \in \{1, \dots, N\}$, where $\vec{v}[n]$ represents the label at time step n .

We now introduce the segmentation process for the sequential vectors used in the model. The core idea is that datasets used to train RNNs-being sequential in nature-can be divided into task-specific segments (e.g., individual sentences in NLP tasks, or fixed-length windows in time series forecasting).

This segmentation enables us to unroll the same RNN unit over these discrete non-overlapping segments and train it in batches, improving both computational efficiency and model generalization. The theoretical foundation for this approach is provided in Proposition 1 of [1], which we will state shortly.²

Let $M < N$ denote the number of segments into which we divide the dataset.

We now describe how the vectors $\vec{s}, \vec{r}, \vec{x}, \vec{y}, \vec{v}$ corresponding to segment $m \in \{0, \dots, M - 1\}$ ³ can each be represented as a vector in $\mathbb{R}^N$, where all entries are zero except for those corresponding to the time steps within segment m .

This allows each segment to be treated independently during computation while maintaining compatibility with the full sequence length.

Definition 2.2.2. For a segment $m \in \{0, \dots, M - 1\}$, we define:

- $j(m)$: the starting index of the m -th segment.
- K_m : the length of the segment.

Therefore, segment m spans the window $n \in [j(m), j(m) + K_m - 1]$.

So now we are able to define the window or the mask vector for each segment.

Definition 2.2.3. For a segment $m \in \{0, \dots, M - 1\}$, we define its corresponding mask function $\vec{w}_m[n]$ as follows:

$$\vec{w}_m[n] = \begin{cases} 1 & , n \in [j(m), j(m) + K_m - 1] \\ 0 & , \text{otherwise} \end{cases}$$

²For a formal proof grounded in probability theory, see the original proposition in [1].

³We adopt zero-based indexing to align with conventions in most programming languages, such as Python.

Using this mask function, we can extract the relevant portion of each vector through element-wise multiplication. This allows us to isolate segment-specific data while maintaining a consistent vector length across the full sequence. The segment-wise representations are summarized in the following table:

Symbol	Meaning	Computed as
$\vec{w}_m[n]$	Window/mask function for segment m	-
$\vec{v}_m[n]$	m -th segment of the output sequence	$\vec{w}_m[n] \cdot \vec{v}[n]$
$\vec{s}_m[n]$	State signal (internal memory) of segment m	$\vec{w}_m[n] \cdot \vec{s}[n]$
$\vec{r}_m[n]$	Readout signal (RNN raw output) of segment m	$\vec{w}_m[n] \cdot \vec{r}[n]$
$\vec{y}_m[n]$	Final observable output of segment m	$\vec{w}_m[n] \cdot \vec{y}[n]$

Table 2.1: Segment-wise representation via masking function $\vec{w}_m[n]$

2.2.2 The Unrolling proposition

The Unrolling Proposition Statement

We will now state a slightly modified 'Proposition 1' from [1] and highlight the most relevant results. For a full proof one can consult the paper.

Proposition 1 ([1]). Given the standard RNN system defined in 1.8, parameterized by $\Theta = \{W_r, W_x, \vec{\theta}_s\}$, assume the existence of a parameter configuration Θ at which the objective function L , for a sequence of length N , is approximately optimal within an acceptable bound.

Suppose there exist non-zero finite constants M and K_m such that $K_m < N$ and $0 \leq m \leq M - 1$. Further, assume that the ground truth output sequence $\vec{v}[n]$ for $0 \leq n \leq N - 1$ can be partitioned into mutually independent segment-level output subsequences $\vec{v}_m[n]$, as defined above.

Then, a single reusable RNN cell, unrolled for an adjustable number of steps K_m , is computationally sufficient both for:

1. Learning a parameter configuration Θ that optimizes L over the training set, and
2. Inferring outputs from unseen input sequences.

Main Results

We begin by introducing the following notation. For a given segment $m \in \{0, \dots, M-1\}$, we denote all vectors with a tilde to represent an index shift by $j(m)$.

This allows us to express the n -th element of a segment more compactly. For example, instead of writing the input signal as $\vec{x}_m[n + j(m)]$, we define:

$$\tilde{\vec{x}}_m[n] := \vec{x}[n + j(m)]$$

This notation simplifies our expressions for time-shifted segment-level vectors in subsequent equations.

We now summarize and present the main results that we can infer from the proof of the proposition.

1. The state and readout signals corresponding to different segments-namely $\vec{s}_m, \vec{r}_m$ and $\vec{s}_n, \vec{r}_n$ for $m \neq n$ -are stochastically independent. It is important to note that this independence does not necessarily extend to the input signals $\vec{x}_m$ and $\vec{x}_n$, which may exhibit statistical dependencies depending on the dataset. Given the independence of the internal states and outputs across segments, we may drop the segment indices when denoting state and readout signals in the remainder of our analysis, treating them as independent realizations of the same process.
2. We prove that all of the RNN can be modeled by this equation.

$$\tilde{\vec{s}}[-1] = \vec{0} \tag{2.1}$$

$$\tilde{\vec{s}}[n] = \begin{cases} W_r \tilde{\vec{r}}[n-1] + W_x \tilde{\vec{x}}_m[n] + \vec{\theta}_s, & \text{if } 0 \leq n \leq K_m - 1 \\ \vec{0}, & \text{otherwise} \end{cases} \tag{2.2}$$

$$\tilde{\vec{r}}[n] = \begin{cases} G_r(\tilde{\vec{s}}[n]), & \text{if } 0 \leq n \leq K_m - 1 \\ \vec{0}, & \text{otherwise} \end{cases} \tag{2.3}$$

$$\tilde{\vec{x}}_m[n] = \begin{cases} \vec{x}[n + j(m)], & \text{if } 0 \leq n \leq K_m - 1 \\ \vec{0}, & \text{otherwise} \end{cases} \tag{2.4}$$

where $0 \leq m \leq M-1$.

3. Equation (2.2) formally demonstrates that the same RNN unit can be reused across segments. By selecting a specific segment m , and unrolling the model for its corresponding length K_m , we obtain a complete description of the RNN's behavior within that segment. This supports the conclusion of the proposition: unrolling the network over each segment's time window is sufficient to achieve a good approximation of the desired function.

Moreover, the reuse of a single RNN unit is reflected in the fact that the internal components—namely the state vector $\vec{s}$ and readout vector $\vec{r}$ —are not segment-dependent and thus do not carry a segment index. In contrast, the input sequence $\vec{x}_m$ remains segment-specific, highlighting the only point of variation between different segments.

This distinction reinforces the earlier result of segment-level independence: the model’s temporal dynamics (as captured by $\vec{s}$ and $\vec{r}$) can be treated as realizations of the same process, conditioned on different input segments.

Chapter 3

Training the RNN: Gradients and Adversarial Behaviors

Having introduced the concept of unrolling in Chapter 2, we now turn our attention to the Backpropagation Through Time (BPTT) algorithm. This method adapts the standard backpropagation algorithm-used in training feedforward networks such as multilayer perceptrons (MLPs)-to the recurrent structure of RNNs by operating on their unrolled form.

By treating the unrolled RNN as a deep feedforward network with shared parameters across time steps, BPTT allows for gradient-based training through the application of the chain rule across time.

We conclude the chapter with illustrating the vanishing/exploding gradient behavior that we introduced in Chapter 1.

At a high level, our goal is to compute the gradient of an error function with respect to the model parameters set Θ , enabling the use of gradient-based optimization algorithms-such as stochastic gradient descent (SGD), Adam, or others-in the standard supervised learning framework.

3.1 The error function and main gradients

Let E denote the objective error function to be minimized (e.g., mean squared error, cross-entropy, etc.).

A natural starting point for the derivation of BPTT is to analyze how the gradient of E propagates through time, beginning with its dependence on the readout signal. Since the readout $\vec{r}[n]$ is a function of the internal state $\vec{s}[n]$, we study how the gradient flows backward through the network by computing the partial derivatives with respect to each time step.

This leads us to examine the gradients of the error function with respect to the state and readout vectors at each time step, i.e., $\frac{\partial E}{\partial \vec{r}[n]}$, $\frac{\partial E}{\partial \vec{s}[n]}$, and how these propagate recursively across time.

We now introduce two key gradient signals that will structure the backward flow of error through the unrolled RNN:

- Let $\chi[n] := \frac{\partial E}{\partial \vec{r}[n]}$ denote the gradient of the error with respect to the readout signal at time step n . This quantity reflects the local sensitivity of the loss to changes in the output at each individual time step. It can be viewed as the output-level gradient, and serves as the entry point for the error signal into the network.
- Let $\psi[n] := \frac{\partial E}{\partial \vec{s}[n]}$ denote the gradient of the error with respect to the state signal (known usually as hidden state) at time step n . This signal plays a central role in BPTT, as it determines how the error propagates through the recurrent structure of the network across time and can be thought of the *global* gradient, since the state signal $s[k]$ affects all of $s[l]$, $l > k$.

Due to the recurrent nature of the architecture, each time step contributes to the final output and, consequently, to the overall prediction error of the segment.

To capture this, we define the total objective error function E as the sum of the individual error terms at each time step.

In typical ML notation we would compare each sample's output with its corresponding ground truth label. In the RNN architecture we can simplify things further based on the fact that the output is deterministically computed through the readout signal through (1.10). So we can define the error function as a function of the ground truths and the errors in regards to the readout signals. So our error function over a specific segment m would be

$$E = \sum_{n=0}^{K_m-1} E(\vec{r}[n], \vec{v}[n])$$

where $E(\cdot)$ is the loss function we defined above and K_m is the length of the segment ¹.

This formulation ensures that the contribution of each time step is accounted for when computing the gradients during backpropagation.

3.2 Backwards DP

In classic Backpropagation fashion we will now derive the recursive equations that allow us to do backwards dynamic programming (DP) on the state gradient $\vec{\psi}$.

¹This formulation is further justified by the fact that the same loss function ℓ is used across all segments.

3.2.1 The Recursive Equations

The equations that allow the gradients to flow are

$$\vec{\chi}[n] = \frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + W_r \vec{\psi}[n+1] \quad (3.1)$$

$$\vec{\psi}[n] = \vec{\chi}[n] \odot \frac{d\vec{G}(\vec{z})}{d\vec{z}} \Big|_{\vec{z}=\vec{s}[n]} = \left(\frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + W_r \vec{\psi}[n+1] \right) \odot \frac{d\vec{G}(\vec{z})}{d\vec{z}} \Big|_{\vec{z}=\vec{s}[n]} \quad (3.2)$$

We begin with deriving (3.1). We wanna outline specifically the truncation process that takes place that is underlying the derivation.

While this sort of truncation might look not intuitive there are two reasons we perform it! Firstly, it serves as a natural extension of the FIR approximating of an IIR. Secondly it allows us to capture a formula of recursive nature where the

Deriving $\vec{\chi}[n]$ For given segment $m = 0, \dots, M-1$ and index $n = 0, 1, \dots, K_m$ we have

$$\begin{aligned} \vec{\chi}[n] &= \sum_{k < n} \frac{\partial E(\vec{r}[k])}{\partial \vec{r}[n]} + \frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + \sum_{k > n} \frac{\partial E(\vec{r}[k])}{\partial \vec{r}[n]} \\ &= \frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + \sum_{k > n} \frac{\partial E(\vec{r}[k])}{\partial \vec{r}[n]}, \text{ since the terms less than } n \text{ are computationally independent of } \vec{r}[n] \\ &\approx \frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + \frac{\partial E(\vec{r}[n+1])}{\partial \vec{r}[n]}, \text{ truncation - keeping only the first term} \end{aligned} \quad (3.3)$$

$$= \frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + \frac{\partial E(\vec{r}[n+1])}{\partial \vec{s}[n+1]} \cdot \frac{\partial \vec{s}[n+1]}{\partial \vec{r}[n]}, \text{ using the chain rule} \quad (3.4)$$

$$= \frac{\partial E(\vec{r}[n])}{\partial \vec{r}[n]} + W_r \cdot \vec{\psi}[n+1], \text{ via the recursive equation for } \vec{s} \quad (3.5)$$

Consequently we naturally derive the equation for the global gradient.

Deriving $\vec{\psi}[n]$

The derivation of the global gradient is grounded in this intuition. The error is affected by the hidden state signal $\vec{s}[n]$ exclusively by the following two quantities.

1. Directly through the present state at n therefore from $\vec{r}[n]$
2. Indirectly from the hidden state that will be formed in $\vec{s}[n+1], \vec{s}[n+2], \dots$
However, due to the recurrent construction we have made - this is a function strictly of $\vec{s}[n+1]$ since it encompasses all the rest of the information.

Therefore via the chain rule we can write

$$\psi[n] = \frac{\partial E}{\partial \vec{r}[n]} \cdot \frac{\partial \vec{r}[n]}{\partial \vec{s}[n]} + \frac{\partial E}{\partial \vec{s}[n+1]} \cdot \frac{\partial \vec{s}[n+1]}{\partial \vec{s}[n]} \quad (3.6)$$

$$= (\chi[n] \odot G'_r(\vec{s}[n]) + (W_r \psi[n+1] + G'_r(\vec{s}[n])), \text{ via the recursive equations} \quad (3.7)$$

$$= (\chi[n] + W_r \psi[n+1]) \odot G'_r(\vec{s}[n]), \text{ factoring out } G' \quad (3.8)$$

We are now in a position to illustrate the vanishing gradient problem in an analytic manner. As stated before, the vanishing gradient problem is the RNN's inherent inability to handle longer-term dependencies. We will now directly illustrate the adversarial gradient flow in long-range dependencies.

Consider two time steps l and n within the same segment such that $l \gg n$. To evaluate how an error signal at a future time step l propagates back to influence a past state gradient at step n , we examine the temporal Jacobian cross-term:

$$\frac{\partial \vec{\psi}[n]}{\partial \vec{\psi}[l]} \quad (3.9)$$

By applying the chain rule across the unrolled computational graph from step l down to step n , we expand this expression into a product of single-step temporal transitions:

$$\frac{\partial \vec{\psi}[n]}{\partial \vec{\psi}[l]} = \frac{\partial \vec{\psi}[n]}{\partial \vec{\psi}[n+1]} \cdot \frac{\partial \vec{\psi}[n+1]}{\partial \vec{\psi}[n+2]} \cdots \frac{\partial \vec{\psi}[l-1]}{\partial \vec{\psi}[l]} \quad (3.10)$$

$$= \prod_{k=n+1}^l \frac{\partial \vec{\psi}[k-1]}{\partial \vec{\psi}[k]} \quad (3.11)$$

To explicitly evaluate the general term of this product, we recall the forward-pass recurrence relation for the hidden state gradient established in Equation (3.2). For any intermediate step k , the local gradient is given by:

$$\vec{\psi}[k-1] = (\vec{\chi}[k-1] + W_r \vec{\psi}[k]) \odot \vec{G}'(\vec{s}[k-1]) \quad (3.12)$$

Differentiating the past gradient state $\vec{\psi}[k-1]$ with respect to its immediate successor $\vec{\psi}[k]$ yields the single-step backward Jacobian matrix $\mathbf{J}_k$. Because $\vec{\chi}[k-1]$ and $\vec{s}[k-1]$ are independent of the downstream gradient $\vec{\psi}[k]$, the first term drops out, leaving:

$$\mathbf{J}_k := \frac{\partial \vec{\psi}[k-1]}{\partial \vec{\psi}[k]} = \text{diag}(\vec{G}'(\vec{s}[k-1])) W_r \quad (3.13)$$

Substituting this localized linear transformation back into our global chain rule product

yields the complete expansion for the long-term gradient trajectory:

$$\frac{\partial \vec{\psi}[n]}{\partial \vec{\psi}[l]} = \prod_{k=n+1}^l \text{diag} \left(\vec{G}'(\vec{s}[k-1]) \right) W_r \quad (3.14)$$

$$= \left[\text{diag} \left(\vec{G}'(\vec{s}[n]) \right) W_r \right] \cdot \left[\text{diag} \left(\vec{G}'(\vec{s}[n+1]) \right) W_r \right] \cdots \left[\text{diag} \left(\vec{G}'(\vec{s}[l-1]) \right) W_r \right] \quad (3.15)$$

We will now show that, the above product becomes effectively the zero matrix, via proving that its norm converges to 0 when the difference $l - n$ becomes arbitrarily big.

Applying the submultiplicative property of matrix norms to the long-term chain rule product yields:

$$\left\| \prod_{k=n+1}^l \mathbf{J}_k \right\|_2 \leq \prod_{k=n+1}^l \|\mathbf{J}_k\|_2 \quad (3.16)$$

To evaluate the norm of the single-step Jacobian matrix, we expand it into its component forms using the diagonal activation derivative and the recurrent weight matrix:

$$\|\mathbf{J}_k\|_2 = \|\text{diag}(\vec{G}'(\vec{s}[k-1])) \cdot W_r\|_2 \quad (3.17)$$

$$\leq \|\text{diag}(\vec{G}'(\vec{s}[k-1]))\|_2 \cdot \|W_r\|_2 \quad (3.18)$$

Since the spectral norm of a diagonal matrix is equal to the maximum absolute value of its diagonal entries, we can bound the first term by the upper limit of the activation function's derivative (for standard activations like tanh, $\max |G'| \leq 1$):

$$\|\mathbf{J}_k\|_2 \leq \max \left| \vec{G}'(\vec{s}[k-1]) \right| \cdot \|W_r\|_2 \quad (3.19)$$

$$\leq 1 \cdot \|W_r\|_2 \quad (3.20)$$

Let $\mu_{\max}$ be the spectral norm of the recurrent weight matrix, $\mu_{\max} := \|W_r\|_2$. Immediately it follows that $\mu_{\max} < 1$ due to the very assumption of stability made all the way in Chapter 1. Substituting this back into the chained product across a temporal distance of $l - n$ steps gives:

$$\left\| \prod_{k=n+1}^l \mathbf{J}_k \right\|_2 \leq (\mu_{\max})^{l-n} \quad (3.21)$$

Taking the limit as the temporal dependency span grows arbitrarily large ($l - n \rightarrow \infty$), we conclude:

$$\lim_{(l-n) \rightarrow \infty} (\mu_{\max})^{l-n} = 0 \quad \blacksquare \quad (3.22)$$

Chapter 4

How LSTMs Learn to Remember

4.1 The LSTM as a solution

As established and proven in the previous chapter, the original RNN architecture suffers from an inability to preserve gradient flow over long sequences-commonly manifesting as either vanishing or exploding gradients.

Several solutions have been proposed to mitigate this issue, as outlined in [5], including leaky units, constrained weight matrices, long delay connections, and echo state networks.

In this section, we focus on one of the most widely adopted and theoretically grounded RNN variants: the Long Short-Term Memory (LSTM) network. Introduced by Hochreiter and Schmidhuber in [6], LSTMs were specifically designed to address the shortcomings of standard RNNs in learning long-term dependencies.^{1 2}

We will analyze the internal structure of the LSTM unit and show how its gated architecture enables more stable and effective gradient propagation during Backpropagation Through Time (BPTT).

4.2 The LSTM unit architecture

In this section we will take a look at the architecture of the LSTM unit and how its innovative design aids in the problems that arise during BPTT.

4.2.1 The sub-units of the LSTM unit

The LSTM unit is inherently more complex than the traditional RNN, as it consists of four subunits, each with their own weights and biases that must be initialized and trained. In the following sections, we describe these subunits and outline how their interaction formulates the entirety of the LSTM.

¹For an intuitive and beginner-friendly walkthrough of LSTM architecture, see Josh Starmer’s visual tutorial.

²There is also the Gated Recurrent Unit architecture (GRUs) as a related simplification that shares many of LSTM’s benefit. However in this paper we focus on the LSTM.

4.2.2 A new memory

The main innovation of the LSTM unit is the introduction of a new kind of memory that enables the model to capture long-term dependencies across time. This memory component is known as the **cell state**, denoted by $\vec{c}_t$. As a result, the LSTM maintains **two distinct types of memory states** - the **cell state** and the **hidden state**, the latter being inherited from the traditional RNN architecture.

The cell state is updated at each time step according to the following equation:

$$\vec{c}_t = \vec{f}_t \odot \vec{c}_{t-1} + \vec{i}_t \odot \vec{g}_t \quad (4.1)$$

where $\vec{f}_t$, $\vec{i}_t$, and $\vec{g}_t$ represent the *forget gate*, *input gate*, and *candidate memory* vectors, respectively. We now describe the role and computation of each of these gates in detail.

4.2.3 The gates

Forget Gate The **forget gate** subunit - introduced by Gers et al. [7] in their aptly titled paper “*Learning to Forget*” - is responsible for determining which parts of the previous cell state should be discarded. As the authors describe, this mechanism is what enables LSTMs to “learn to remember” by selectively forgetting irrelevant information from the past.

At each time step t , the forget gate transforms the previous hidden state and the current input through a learned linear transformation followed by a sigmoid activation:

$$\vec{f}_t = \sigma(W_{fh}\vec{h}_{t-1} + W_{fx}\vec{x}_t + \vec{b}_f) \quad (4.2)$$

The output $\vec{f}_t \in (0, 1)^d$ contains one gating coefficient per memory dimension, which is then used to scale the previous cell state $\vec{c}_{t-1}$ element-wise.

Input Gate and Candidate Memory The **input gate** determines how much new information should be added to the cell state, while the **candidate memory vector** proposes the actual content of that update. Together, they form the mechanism by which fresh information enters long-term memory.

The input gate applies a sigmoid activation to a learned transformation of the current input and previous hidden state:

$$\vec{i}_t = \sigma(W_{ih}\vec{h}_{t-1} + W_{ix}\vec{x}_t + \vec{b}_i) \quad (4.3)$$

$$\vec{g}_t = \tanh(W_{gh}\vec{h}_{t-1} + W_{gx}\vec{x}_t + \vec{b}_g) \quad (4.4)$$

The output $\vec{i}_t \in (0, 1)^d$ gates how much of the candidate content $\vec{g}_t \in \mathbb{R}^d$ is incorporated into the cell state at this time step.

Output Gate and Hidden State The **output gate** controls how much of the updated cell state should be exposed as output - that is, how much of the long-term memory is written into the short-term *hidden state* $\vec{h}_t$.

It is computed as follows:

$$\vec{o}_t = \sigma(W_{oh}\vec{h}_{t-1} + W_{ox}\vec{x}_t + \vec{b}_o) \quad (4.5)$$

$$\vec{h}_t = \vec{o}_t \odot \tanh(\vec{c}_t) \quad (4.6)$$

The hidden state $\vec{h}_t$ is then used either as output (e.g., in sequence modeling tasks) or passed to the next LSTM unit at time $t + 1$. This architecture allows gradients to flow more effectively during backpropagation through time (BPTT), mitigating the vanishing gradient problem that affects vanilla RNNs.

This final LSTM unit cell can be intuitively visualized as a more complex RNN computational unit.

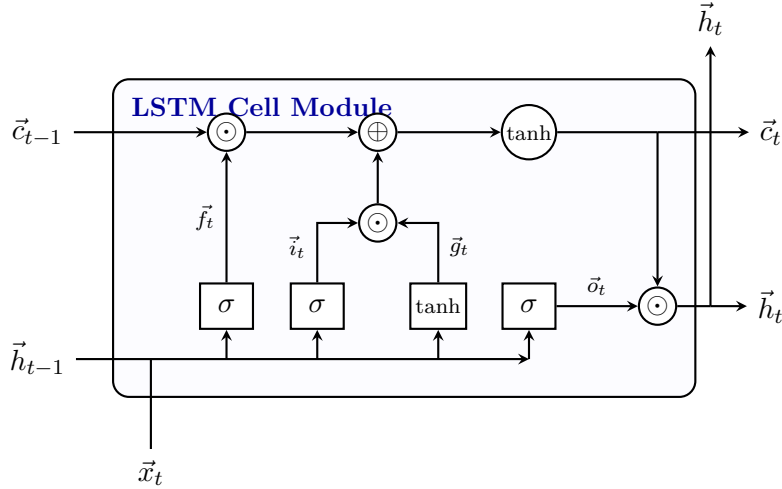


Figure 4.1: Detailed internal schematic of the Long Short-Term Memory (LSTM) block, mapping the structural interaction of the forget, input, and output gates with the long-term cell state rail.

The unrolling paradigm applies in the LSTM unit just like the RNN, despite the more intricate inner build.

Chapter 5

Empirical Validation - Proof of Concept

5.1 Introduction and Experimental Setup

In the above chapters, we have mathematically established the theoretical foundation of the base RNN architecture and its inherent susceptibility to the vanishing gradient problem in long term dependency settings.

In this chapter, we aim to examine that analytical framework that was covered on real datasets, curated specifically tailored to stress test the RNN. We examine two distinct environments: a real-world time-series forecasting task (weather forecasting in Melbourne using past data) and a strict synthetic stress test known as 'the adding problem' designed to isolate long-term memory capabilities created by the very inventors of the LSTM, as outlined in the paper that introduced the architecture [6].

The complete experiment including the data pipelines, model architectures, and training logs, is available here: https://github.com/stavroszd/Deep_Learning_Time_Series_PoC

5.2 Case Study I: The Long-Window Trap in Real-World Time Series

To test this, our first experiment utilizes the Daily Minimum Temperatures dataset (Melbourne, Australia), consisting of 10 years of daily recordings.

5.2.1 Experimental Design

We establish a lookback window of 365 days to predict the temperature of the immediate next day ($T + 1$).

Theoretically, a sequence length of 365 timesteps is well beyond the gradient stability limit of an RNN. Initially then, one might expect the LSTM to drastically outperform

the standard RNN by leveraging its gated memory to recall the exact temperature from a year earlier.

5.2.2 Results and Discussion

However, the empirical results present a counter-intuitive phenomenon often encountered in production data science.

Architecture	Final Training MAE	Final Validation MAE	Epochs to Converge
SimpleRNN	0.0734	0.0662	28
LSTM	0.0733	0.066	99

Table 5.1: Performance metrics for the 365-day lookback temperature forecasting task.

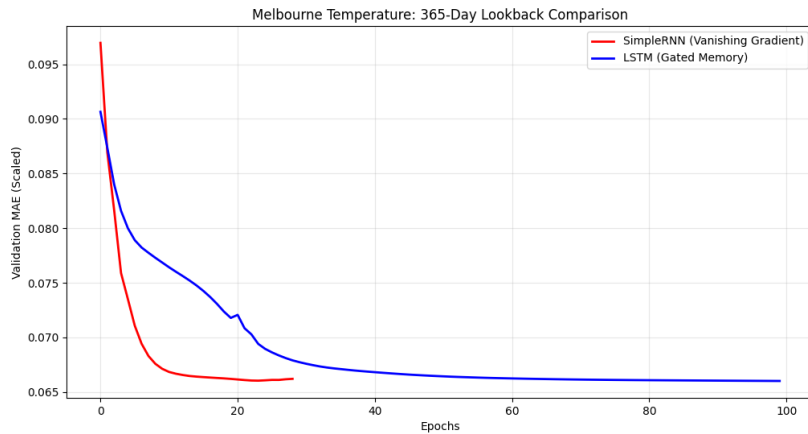


Figure 5.1: Validation MAE over epochs. Notice the initial convergence speed of the SimpleRNN compared to the LSTM.

As shown in Figure 5.1, the RNN outperforms the LSTMs. Not only is it able to train and achieve a very good MAE metric, it also does it in a significantly faster manner converging at around 25 epochs contrary to the 80-100 range of its LSTM counter part.

This might come across as suprising in a first read. However, a great principle that permeates the practice of Data Science is illuminated once again. 'Understand the data before you fit a model'. That very lesson is a telltale sign of maturity of a Data Science practitioner.

In this case, despite the very large input window, the true Data Generating Process (DGP) for tomorrow's temperature is heavily weighted by the immediate short-term history (the last 1-3 days).

Despite being fed a 365-day sequence, the RNN ironically enough, through ignoring the

distant past and optimizing for the local, short-term correlation is able to learn a better representation.

Conversely, the LSTM’s robust representational capacity becomes a temporary computational burden. With four interconnected interacting layers (the input, forget, and output gates, plus the candidate cell state), the LSTM requires significantly more gradient updates (which means more data) to calibrate its representations.

Pragmatically speaking, how might one apply that great principle we mentioned? A mature data scientist, wanting to understand the DGP governing the dataset would naturally be curious around the underlying temporal dependencies. A great tool found in the statistics paradigm of time series forecasting, would be the estimation of the autocorrelation function (ACF) and of the partial autocorrelation function (PACF). 5.2 5.3

Naturally, the plot reveals that the temperature is highly correlated with the last few days, with around 75% correlation for the first few and around 50% for the months above.

The seasonality component is also highlighted by the fact that very strong positive correlation is shown around the 1 year mark (lags 360) and very strong negative correlation is shown at around the 6 month mark, very logical as the 6-month delta implies a warm season-cold season switch (fall,winter to spring,summer and vice versa).

The PACF makes an even stronger case for the above hypothesis as it’s functionality is to measure the correlations removing all the effects of the in between lags.

In summary, despite the long window, the dependency is short-term based (as shown with the statistical estimators of correlation) and the RNN is able to capture it way faster.

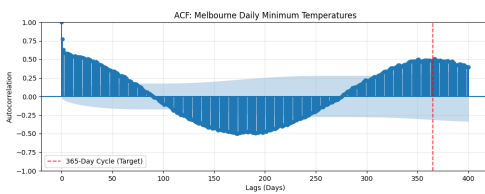


Figure 5.2: Autocorrelation Function (ACF) of Melbourne Daily Minimum Temperatures.

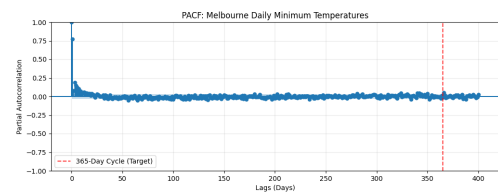


Figure 5.3: Partial Autocorrelation Function (PACF) of Melbourne Daily Minimum Temperatures.

5.3 Case Study II: The Adding Problem

While Case Study I demonstrates that real-world noise and short-term correlations can mask the vanishing gradient problem, we require a strict environment to prove that the LSTM’s long term memory functions as theorized. For this, we turn to the foundational “Adding Problem,” originally introduced by Hochreiter and Schmidhuber.

5.3.1 The Data Generating Process

The Adding Problem strips away local correlations entirely. The model is fed a sequence of length $L = 200$. Each timestep contains two features: a continuous random variable drawn from $\mathcal{U}(0, 1)$, and a binary mask. The mask contains exactly two 1s located at random indices across the 200 steps, with all other values set to 0. The network is tasked with outputting the exact sum of the two random continuous variables indicated by the mask.

To succeed, the network *must* identify the first flagged number and carry its exact value through up to 198 timesteps of pure mathematical noise without degradation.

5.3.2 Breaking the Plateau

This task isolates the vanishing gradient limitation perfectly. For a SimpleRNN, retaining a specific continuous value across 200 state updates involves 200 sequential matrix multiplications. Mathematically, the gradient signal decays exponentially, rendering the early sequence features invisible during backpropagation.

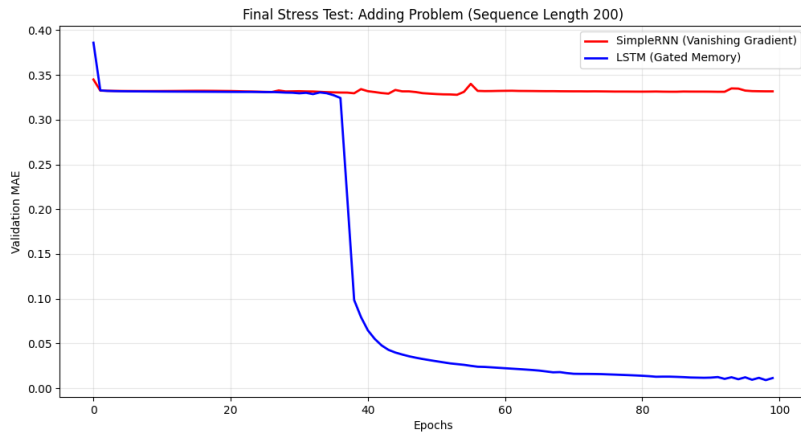


Figure 5.4: Validation MAE on the Adding Problem ($L = 200$). The SimpleRNN remains trapped at baseline error, while the LSTM successfully synchronizes its gates to solve the sequence.

As illustrated in Figure 5.4, the SimpleRNN immediately plateaus at an error rate consistent with predicting the statistical average of the sum. It fails to learn the sequence mapping entirely.

In stark contrast, the LSTM exhibits a profound phase transition. While it initially mirrors the RNN’s plateau as it optimizes in the weight space, a sharp, sudden decrease in error occurs (typically around Epoch 30). This drop visually represents the exact epoch where the LSTM’s input and forget gates successfully synchronized.

By opening the input gate at the first marker and closing the forget gate for the remainder of the sequence, the LSTM shielded the target value inside its cell state, allowing the

uncorrupted gradient to flow backward through time.

5.4 Conclusion

The empirical findings validate the mathematical derivations presented in the preceding chapters. Standard RNNs offer computational efficiency for short-horizon forecasting but suffer structural failure when forced to bridge wide temporal gaps. The LSTM, by introducing a gated memory mechanism, successfully overrides the vanishing gradient, proving indispensable for data generating processes driven by true, long-term dependencies.

The final caveat remains. The mature data scientist must possess the ability to reason about datasets and their corresponding DGPs through fundamental statistic principles in order to choose the appropriate deep learning architecture.

Bibliography

- [1] A. Sherstinsky, “Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network,” *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, Mar. 2020.
- [2] I. D. Mienye, T. G. Swart, and G. Obaido, “Recurrent neural networks: A comprehensive review of architectures, variants, and applications,” *Information*, vol. 15, no. 9, 2024.
- [3] D. Griffiths and D. Higham, *Numerical Methods for Ordinary Differential Equations*. Springer Undergraduate Mathematics Series, Springer, 2010.
- [4] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Cham, Switzerland: Springer International Publishing, 2018.
- [5] B. Ghojogh and A. Ghodsi, “Recurrent neural networks and long short-term memory networks: Tutorial and survey,” 2023.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] F. A. Gers, J. Schmidhuber, and F. Cummins, “Learning to forget: Continual prediction with lstm,” in *Proceedings of the 9th International Conference on Artificial Neural Networks (ICANN 2000)*, pp. 850–855, IET, 2000.